Python Interview Questions and Answers

Simple answers you can say in your own words
AI / ML Engineer interviews

How to use this: Read each question, answer out loud first, then compare with the answer here. The 'Say in interview' box is a short version you can speak in 20 to 30 seconds. Try to type the small code examples yourself, because interviewers often ask you to write code. These are practice questions based on common topics, not leaked questions from any company.

Part 1: Basics

Q1. What is Python? What are its main features?

Answer:

Python is a high-level, general-purpose programming language. It is easy to read and write, so it is the most popular language for AI and data work.

- **Interpreted:** the code runs line by line, with no separate compile step.
- **Dynamically typed:** we do not declare the type of a variable. Python finds it at run time.
- **Object-oriented** and also supports functional style.
- Big library support: numpy, pandas, PyTorch, and many more.

Say in interview: Python is an easy, interpreted, dynamically typed language with a huge library ecosystem, which is why it is the main language for AI.

Q2. What is the difference between mutable and immutable types?

Answer:

A mutable object can be changed after it is created. An immutable object cannot be changed. If we try to change it, Python makes a new object.

- **Mutable:** list, dictionary, set.
- **Immutable:** int, float, string, tuple, frozenset, bool.
- Only immutable objects can be used as dictionary keys (they must be hashable).

```
s = "hello"
# s[0] = "H"      -> error, strings are immutable
nums = [1, 2, 3]
nums[0] = 99      # works, lists are mutable
```

Say in interview: Mutable objects like lists and dictionaries can be changed in place. Immutable ones like strings and tuples cannot.

Q3. What is the difference between list, tuple, set, and dictionary?

Answer:

They are the four main built-in collections.

- **List:** ordered, can change, allows duplicates. Written as [1, 2, 3].
- **Tuple:** ordered, cannot change, allows duplicates. Written as (1, 2, 3). Faster and safe for fixed data.
- **Set:** no order, no duplicates, very fast to check if an item is present. Written as {1, 2, 3}.

- **Dictionary:** key and value pairs, keys are unique, very fast lookup by key. Written as {'a': 1}. Since Python 3.7 it keeps the insertion order.

Say in interview: List is changeable and ordered, tuple is fixed, set has only unique items, and dictionary stores key and value pairs.

Q4. What is a list comprehension?

Answer:

It is a short way to create a list from another list or a loop, in one line. It is shorter and usually faster than a normal for loop.

```
squares = [x * x for x in range(5)]            # [0, 1, 4, 9, 16]
evens = [x for x in range(10) if x % 2 == 0]   # [0, 2, 4, 6, 8]
d = {x: x * x for x in range(3)}               # dict comprehension
```

Say in interview: A list comprehension builds a new list in one line, from a loop and an optional condition.

Q5. What is the difference between == and is?

Answer:

== checks if the two values are equal. 'is' checks if both names point to the exact same object in memory.

```
a = [1, 2]
b = [1, 2]
a == b     # True, same values
a is b     # False, two different objects
x = None
x is None  # the right way to check None
```

Say in interview: == compares the values. 'is' checks if it is the same object. I use 'is' mainly for None.

Q6. What is the difference between shallow copy and deep copy?

Answer:

A shallow copy makes a new outer object, but the inner objects are still shared with the original. A deep copy copies everything inside also, so the two are fully separate.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)        # shallow
c = copy.deepcopy(a)    # deep
a[0][0] = 99
print(b[0][0])   # 99, inner list is shared
print(c[0][0])   # 1, fully separate
```

Say in interview: Shallow copy shares the inner objects. Deep copy makes a fully independent copy.

Q7. What are *args and **kwargs?

Answer:

They let a function take any number of arguments. *args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dictionary.

```
def show(*args, **kwargs):
    print(args)     # (1, 2)
    print(kwargs)   # {'name': 'Ravi'}

show(1, 2, name="Ravi")
```

Say in interview: *args takes any number of positional arguments as a tuple. **kwargs takes any number of named arguments as a dictionary.

Q8. What is the mutable default argument problem?

Answer:

A default value is created only once, when the function is defined, not every time it is called. If the default is a list or dictionary, the same object is reused across calls, and it keeps growing. The fix is to use None as the default and create the list inside the function.

```
def add(item, items=[]):       # wrong
    items.append(item)
    return items

add(1)   # [1]
add(2)   # [1, 2]  surprise!

def add_ok(item, items=None):  # right
    if items is None:
        items = []
    items.append(item)
    return items
```

Say in interview: Default values are created once, so a list default is shared between calls. I use None and create the list inside the function.

Part 2: Functions and OOP

Q9. What is a decorator?

Answer:

A decorator is a function that takes another function, adds some extra behavior before or after it, and returns it, without changing the original code. We apply it with the @ symbol. Common uses: logging, timing, checking login, retrying, and caching.

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print("took", time.time() - start)
        return result
    return wrapper

@timer
def work():
    time.sleep(1)
```

Say in interview: A decorator wraps a function to add extra behavior, like timing or logging, without changing the function itself.

Q10. What are generators and iterators? What does yield do?

Answer:

An iterable is anything we can loop over, like a list. An iterator gives one item at a time. A generator is a simple way to make an iterator using a function with yield. It produces values one by one, only when asked, and does not keep everything in memory. So it is good for big data, like reading a huge file line by line.

```
def count_up(n):
    for i in range(1, n + 1):
        yield i

for x in count_up(3):
    print(x)      # 1, 2, 3
```

Say in interview: A generator uses yield to give values one at a time without storing them all in memory, so it is good for big data.

Q11. What are lambda, map, and filter?

Answer:

A lambda is a small one-line function without a name. map applies a function to every item. filter keeps only the items that pass a condition. Today, list comprehensions are often preferred because they are easier to read.

```
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))        # [2, 4, 6, 8]
evens = list(filter(lambda x: x % 2 == 0, nums))  # [2, 4]
```

Say in interview: A lambda is a small unnamed function. map transforms every item and filter keeps the items that match a condition.

Q12. What is variable scope? What is LEGB?

Answer:

Python looks for a variable in this order: Local (inside the function), Enclosing (the outer function), Global (the file level), and Built-in (like len and print). To change a global variable inside a function we need the global keyword. For an outer function's variable, we use nonlocal.

Say in interview: Python searches for a name in Local, Enclosing, Global, then Built-in scope. That order is LEGB.

Q13. Explain the four pillars of OOP in Python.

Answer:

OOP means we organize code into classes (the design) and objects (the real things made from the design).

- **Encapsulation:** keep data and the methods together, and hide the details. In Python, a name starting with an underscore means 'private by convention'.
- **Inheritance:** a child class gets the features of a parent class.
- **Polymorphism:** the same method name works differently in different classes.
- **Abstraction:** show only what is needed, and hide the complex part. We use abstract classes for this.

Say in interview: OOP organizes code in classes and objects. The four pillars are encapsulation, inheritance, polymorphism, and abstraction.

Q14. What is the difference between instance method, class method, and static method?

Answer:

They differ in what they receive and what they can access.

- **Instance method:** gets self, and works with one object's data.
- **Class method:** gets cls, and works with the class itself. Often used as an alternative constructor.
- **Static method:** gets nothing automatically. It is just a normal function placed inside the class.

```
class Employee:
    company = "ABC"
    def __init__(self, name):
        self.name = name
    def greet(self):                  # instance method
        return "Hi " + self.name
    @classmethod
    def from_string(cls, s):          # class method
        return cls(s.split("-")[0])
    @staticmethod
    def is_valid(name):               # static method
        return len(name) > 0
```

Say in interview: Instance methods use self, class methods use cls, and static methods use neither.

Q15. What is __init__ and self? What is the difference between __str__ and __repr__?

Answer:

__init__ is the constructor. Python calls it automatically when we create an object, to set the starting values. self is the object itself, and it lets the methods reach the object's data. __str__ gives a readable text for users, used by print. __repr__ gives a detailed text for developers, used for debugging.

Say in interview: __init__ sets up a new object and self is that object. __str__ is for readable output and __repr__ is for developers.

Q16. What is multiple inheritance and MRO?

Answer:

A class can inherit from more than one parent class. If both parents have the same method, Python needs to know which one to use. The MRO (Method Resolution Order) is the order in which Python searches the classes. We can see it with ClassName.mro(). super() follows this same order.

Say in interview: Multiple inheritance means more than one parent. MRO is the order in which Python searches the parents for a method.

Q17. How does exception handling work in Python?

Answer:

We put risky code in a try block, and handle the error in except. else runs if there was no error. finally always runs, whether there was an error or not, so it is used for cleanup. We can raise our own errors with raise, and make custom exceptions by inheriting from Exception.

```
try:
    x = int("abc")
except ValueError as e:
    print("bad value:", e)
else:
    print("no error")
finally:
    print("always runs")
```

Say in interview: try holds the risky code, except handles the error, else runs if no error, and finally always runs for cleanup.

Q18. What is a context manager? What does the 'with' statement do?

Answer:

A context manager sets up something at the start and cleans it up at the end, automatically, even if an error happens. The most common use is opening a file. We do not need to remember to close it. Other uses: database connections and locks.

```
with open("data.txt") as f:
    text = f.read()
# the file is closed automatically here
```

Say in interview: The with statement handles setup and cleanup automatically, like closing a file even if an error occurs.

Part 3: Advanced Topics

Q19. What is the GIL? Threading vs multiprocessing vs asyncio?

Answer:

The GIL (Global Interpreter Lock) in standard Python allows only one thread to run Python code at a time. So threads do not give a speed gain for heavy CPU work.

- **Threading:** good for I/O waiting, like network calls or files.
- **Multiprocessing:** separate processes, each with its own GIL. Good for heavy CPU work.

- **Asyncio:** one thread that switches between tasks while they wait. Good for thousands of network calls, like calling an LLM API many times.

Say in interview: The GIL lets only one thread run Python code at a time. So I use multiprocessing for CPU work, and threads or asyncio for waiting on I/O.

Q20. What is async and await? Why is it useful for LLM applications?

Answer:

async and await let one program start many slow tasks, like API calls, and not sit idle while waiting. When one task waits, Python runs another. For LLM apps, we often call an API many times, and each call takes seconds. Using asyncio, many calls run together, so the total time is close to one call, not the sum of all.

```
import asyncio

async def call_api(i):
    await asyncio.sleep(1)     # pretend this is an API call
    return i

async def main():
    results = await asyncio.gather(
        call_api(1), call_api(2), call_api(3))
    print(results)             # takes about 1 second, not 3

asyncio.run(main())
```

Say in interview: Async lets many slow API calls wait together instead of one after another, so the total time is much shorter.

Q21. How does Python manage memory? What is garbage collection?

Answer:

Python uses reference counting. Each object counts how many names point to it. When the count becomes zero, the memory is freed. This fails for circular references (A points to B, and B points to A), so Python also has a garbage collector that finds and cleans such cycles.

Say in interview: Python counts references to each object and frees it at zero. A garbage collector also cleans the circular references.

Q22. What are type hints, dataclasses, and Pydantic?

Answer:

Type hints tell the expected type of a variable or function, like name: str. Python does not enforce them, but editors and tools use them to find bugs. A **dataclass** is a short way to write a class that mainly stores data. **Pydantic** is a popular library that checks the data types at run time. It is widely used in AI work, for example to validate the structured JSON output of an LLM, and in FastAPI.

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int

u = User("Ravi", 25)
```

Say in interview: Type hints document the types, a dataclass makes data classes with less code, and Pydantic validates data at run time, which is useful for LLM outputs.

Q23. What are virtual environments and pip?

Answer:

A virtual environment is a separate Python setup for one project, with its own packages. It stops version conflicts between projects. pip is the tool to install packages. We list the packages of a project in a requirements.txt file, so others can install the same versions.

```
python -m venv venv
source venv/bin/activate      # on Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Say in interview: A virtual environment keeps each project's packages separate, and pip installs them. requirements.txt records the versions.

Q24. How do you read and write files and JSON in Python?

Answer:

We use open() with the with statement. The modes are 'r' for read, 'w' for write (overwrites), and 'a' for append. For JSON, we use the json module: json.load reads from a file, json.dump writes to a file, json.loads converts a string to Python data, and json.dumps converts Python data to a string.

```
import json

with open("data.json") as f:
    data = json.load(f)          # file -> dict

with open("out.json", "w") as f:
    json.dump(data, f, indent=2) # dict -> file
```

Say in interview: I use open with the with statement for files, and the json module to convert between JSON and Python dictionaries.

Part 4: NumPy and Pandas

Q25. Why is NumPy faster than a Python list? What is broadcasting?

Answer:

A NumPy array stores items of one type together in memory, and the calculations run in fast compiled code. A Python list stores separate objects of any type, so loops are slower. Broadcasting means NumPy can do math between arrays of different shapes by stretching the smaller one automatically, for example adding one number to a whole array.

```
import numpy as np
a = np.array([1, 2, 3])
print(a * 2)      # [2 4 6], no loop needed
print(a + 10)     # [11 12 13], broadcasting
```

Say in interview: NumPy arrays are stored compactly and use fast compiled code, so they are much faster than lists. Broadcasting lets arrays of different shapes work together.

Q26. What is Pandas? What is the difference between loc and iloc?

Answer:

Pandas is the main library for table data. A DataFrame is a table with rows and columns, and a Series is one column. loc selects by label (names), and iloc selects by position (numbers).

```
import pandas as pd
df = pd.DataFrame({"name": ["A", "B", "C"], "age": [25, 32, 41]})

df.loc[df["age"] > 30, "name"]   # by condition and label
df.iloc[0, 1]                    # first row, second column
```

Say in interview: Pandas handles table data with DataFrames. loc selects by label and iloc selects by position.

Q27. How do you handle missing values, group, and merge in Pandas?

Answer:

These are the most common data tasks:

- **Missing values:** isna().sum() to count them, dropna() to remove, and fillna() to fill with a value like the mean.
- **Group:** groupby splits the data by a column and applies a calculation like mean or sum.
- **Merge:** merge joins two tables on a common column, like a SQL join.

```
df.isna().sum()                          # count missing per column
df["age"] = df["age"].fillna(df["age"].mean())
df.groupby("city")["salary"].mean()      # average salary per city
pd.merge(orders, users, on="user_id", how="left")
```

Say in interview: I use isna, dropna, and fillna for missing values, groupby for summaries, and merge to join tables.

Part 5: Common Coding Questions

Q28. How do you reverse a string, check a palindrome, and count word frequency?

Answer:

These are very common warm-up questions. Slicing with [::-1] reverses a string. A palindrome reads the same in both directions. Counter from the collections module counts items easily.

```
s = "madam"
print(s[::-1])            # reverse the string
print(s == s[::-1])       # True, it is a palindrome

from collections import Counter
text = "to be or not to be"
print(Counter(text.split()))   # {'to': 2, 'be': 2, 'or': 1, 'not': 1}
```

Say in interview: I use slicing to reverse, compare with the reversed string for a palindrome, and Counter to count words.

Q29. How do you find duplicates in a list, and check if two words are anagrams?

Answer:

A set remembers what we have already seen, and checking a set is very fast. Two words are anagrams if their sorted letters are the same.

```
def find_duplicates(items):
    seen, dups = set(), set()
    for x in items:
        if x in seen:
            dups.add(x)
        seen.add(x)
    return list(dups)

def is_anagram(a, b):
    return sorted(a) == sorted(b)
```

Say in interview: I use a set to track seen items for duplicates, and compare the sorted letters for anagrams.

Q30. Two Sum: given a list and a target, find two numbers that add up to the target.

Answer:

The slow way checks every pair, which takes O(n squared) time. A better way is to use a dictionary. For each number, we check if (target minus the number) was already seen. If yes, we found the pair. It takes one pass, so the time is O(n).

```
def two_sum(nums, target):
    seen = {}                       # value -> index
    for i, n in enumerate(nums):
        need = target - n
        if need in seen:
            return [seen[need], i]
        seen[n] = i

print(two_sum([2, 7, 11, 15], 9))   # [0, 1]
```

Say in interview: I loop once and keep a dictionary of the numbers I have seen. For each number I check if the needed partner is already there. It is O(n) time.

Part 6: Python for LLM and AI Work

Q31. How do you call an LLM API safely and reliably from Python?

Answer:

Calling an API in a real project needs more than one line of code:

- Keep the API key in an environment variable, never in the code or in Git.
- Use the official SDK or the requests library, and always set a timeout.
- Handle errors and rate limits, and retry with exponential backoff (wait longer after each failure).
- Validate the output, for example check that the JSON is valid with Pydantic.
- Log the tokens used, the cost, and the delay.
- Use async calls or batching when you have many requests.

Say in interview: I keep the key in an environment variable, set timeouts, retry with backoff, validate the output, and log the cost.

Q32. How do you write a retry with exponential backoff?

Answer:

If a call fails, we wait a short time, then try again. Each time we wait longer (1 second, 2 seconds, 4 seconds). We stop after a few tries and raise the error. This is very useful for rate limit errors and temporary network failures.

```
import time

def call_with_retry(fn, tries=3):
    for i in range(tries):
        try:
            return fn()
        except Exception:
            if i == tries - 1:
                raise                # last try, give up
            time.sleep(2 ** i)       # wait 1s, then 2s, then 4s
```

Say in interview: On failure I wait, then retry, doubling the wait each time, and give up after a few tries.

If you do not know an answer, say honestly: 'I have not worked on this directly, but my understanding is...' and then explain what you know. It is better than guessing.